

Dunaújvárosi Egyetem Bánki Donát Technikum

Projekt feladat dokumentáció

Tartalom

Az ötlet rövid leírása:	2
Arduino kód.....	2
index.html kód	3
A projektről készült képek.....	3
Fejlesztési lehetőségek	7
Önreflexió	8

Tantárgy neve: IoT

Projekt tervezői: Sümegi Tamás

Projekt címe: Hőmérséklet-páratartalom Web Plotter

Osztály: 13.B

Az ötlet rövid leírása:

A projekt során egy ESP8266 mikrokontrollerre épülő webes megjelentető rendszert készítettem, amely valós időben méri és WebSocketen keresztül egy böngészőben megjeleníti a hőmérséklet és páratartalom értékeket. A rendszer alapját a Newbiely ESP8266 Web Plotter példája adta, amelyet kiegészítettem DHT-szenzor támogatással és kétváltozós grafikus megjelenítéssel. A felhasználó így egy egyszerű weboldalon élőben követheti a környezeti adatok változását.

Arduino kód

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <WebSocketsServer.h>
#include "index.h"
#include <DHT.h>
const char* ssid = "HONOR Magic8 Lite";
const char* password = "Tomi2006";
#define DHTPIN 2
#define DHTTYPE DHT22
ESP8266WebServer server(80);
WebSocketsServer plotter = WebSocketsServer(81);
DHT dht(DHTPIN, DHTTYPE);
unsigned long last_update = 0;
const unsigned long UPDATE_INTERVAL = 2000;
void setup() {
    Serial.begin(115200);
    delay(500);
    dht.begin();
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.println("Connecting to WiFi...");
    }
    Serial.println("Connected to WiFi");
    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
    plotter.begin();
    server.on("/", HTTP_GET, []() {
        server.send(200, "text/html", HTML_CONTENT);
    });
    server.begin();
    Serial.println("HTTP server started");
}
void loop() {
    server.handleClient();
    plotter.loop();
    unsigned long now = millis();
    if (now - last_update > UPDATE_INTERVAL) {
```

```

last_update = now;
float humidity = dht.readHumidity();
float temperature = dht.readTemperature();
// Hibás olvasás esetén
if (isnan(humidity) || isnan(temperature)) {
    Serial.println("Failed to read from DHT sensor!");
    return;
}
// Serial Plotter
Serial.print(temperature, 2);
Serial.print(" ");
Serial.println(humidity, 2);

// Serial Monitor
Serial.print("Temp: "); Serial.print(temperature, 2);
Serial.print(" °C\tHum: "); Serial.println(humidity, 2);

// Websocket szerver
String payload = String(temperature, 2) + "\t" + String(humidity, 2);
plotter.broadcastTXT(payload);
}
}

```

index.html kód

```

const char *HTML_CONTENT = R"=====(
<!DOCTYPE html>
<html>
<head>
    <title>ESP8266 DHT22 Monitor</title>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Arial,
sans-serif; text-align: center; color: #333; margin: 20px; }
        h1 { font-weight: 700; margin-bottom: 5px; font-size: 24px; }
        .readout-container { margin-bottom: 15px; }
        .readout { font-size: 26px; margin: 8px 0; font-weight: 400; }
        #chart-wrapper {
            position: relative;
            margin: auto;
            height: 250px;
            width: 95%;
            max-width: 500px;
        }
        .controls { margin-top: 20px; font-size: 0.8em; color: #888; }
        button { padding: 5px 15px; border-radius: 4px; border: 1px solid #ddd; background:
#fff; cursor: pointer; }
    </style>
)====="

```

```

</head>
<body>
  <h1>ESP8266 DHT22 Monitor</h1>
  <div class="readout-container">
    <div class="readout">Temperature: <span id="curr_temp">--.--</span> &deg;C</div>
    <div class="readout">Humidity: <span id="curr_hum">--.--</span> %</div>
  </div>
  <div id="chart-wrapper">
    <canvas id="myChart"></canvas>
  </div>
  <div class="controls">
    <button id="btn_connect">Connect</button>
    <p>Status: <span id="ws_state">CLOSED</span></p>
  </div>
<script>
  var websocket;
  var maxDataPoints = 15;
  const ctx = document.getElementById('myChart').getContext('2d');
  const myChart = new Chart(ctx, {
    type: 'line',
    data: {
      labels: [],
      datasets: [{
        // Fixed degree symbol in the Chart Legend here
        label: 'Temperature (°C)',
        data: [],
        borderColor: '#FF3B30',
        backgroundColor: '#FF3B30',
        borderWidth: 2,
        pointRadius: 3,
        tension: 0.3,
        yAxisID: 'y',
      }, {
        label: 'Humidity (%)',
        data: [],
        borderColor: '#007AFF',
        backgroundColor: '#007AFF',
        borderWidth: 2,
        pointRadius: 3,
        tension: 0.3,
        yAxisID: 'y1',
      }
    ],
    options: {
      responsive: true,
      maintainAspectRatio: false,
      scales: {
        y: { type: 'linear', display: true, position: 'left', min: 20, max: 40 },
        y1: { type: 'linear', display: true, position: 'right', min: 0, max: 100, grid: {
drawOnChartArea: false } }

```

```

    },
    plugins: {
        legend: { position: 'top' }
    }
}
});
function updateUI(temp, hum) {
    document.getElementById("curr_temp").innerText = temp.toFixed(2);
    document.getElementById("curr_hum").innerText = hum.toFixed(2);

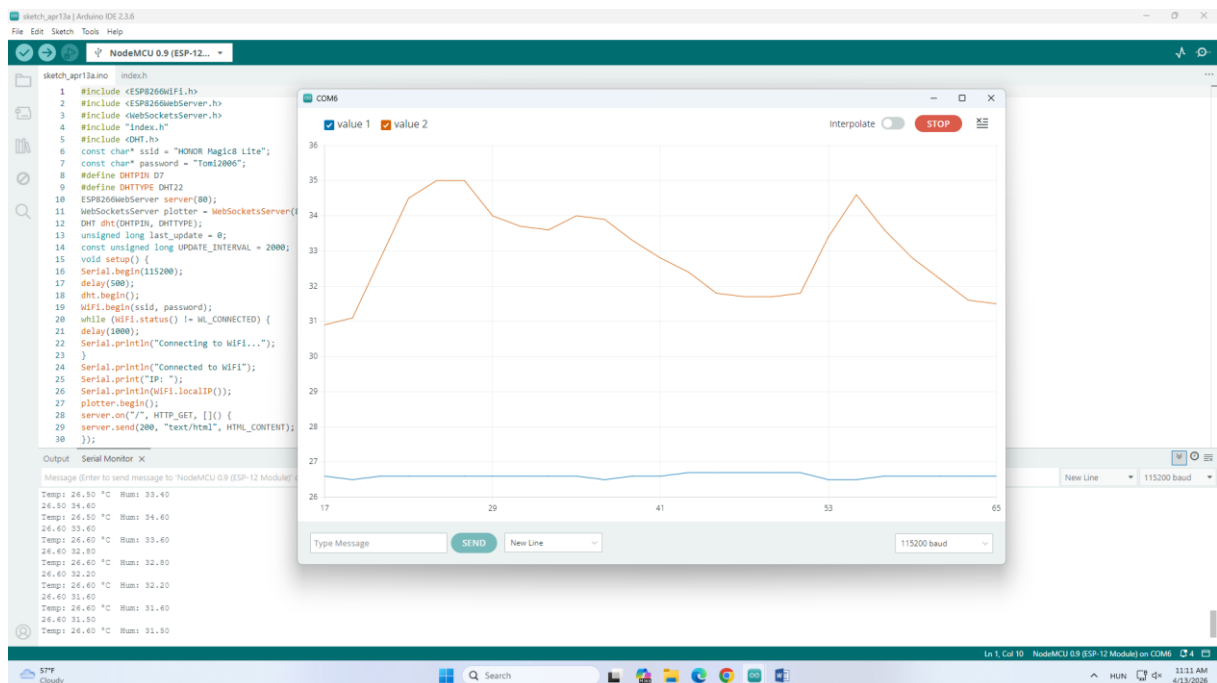
    let now = new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit', second:
'2-digit' });
    myChart.data.labels.push(now);
    myChart.data.datasets[0].data.push(temp);
    myChart.data.datasets[1].data.push(hum);
    if (myChart.data.labels.length > maxDataPoints) {
        myChart.data.labels.shift();
        myChart.data.datasets[0].data.shift();
        myChart.data.datasets[1].data.shift();
    }
    myChart.update('none');
}
function connectWS() {
    if (!websocket) {
        websocket = new WebSocket("ws://" + window.location.hostname + ":81");
        document.getElementById("ws_state").innerText = "CONNECTING...";
        websocket.onopen = () => {
            document.getElementById("ws_state").style.color = "green";
            document.getElementById("ws_state").innerText = "CONNECTED";
            document.getElementById("btn_connect").innerText = "Disconnect";
        };
        websocket.onclose = () => {
            document.getElementById("ws_state").style.color = "grey";
            document.getElementById("ws_state").innerText = "CLOSED";
            document.getElementById("btn_connect").innerText = "Connect";
            websocket = null;
        };
        websocket.onmessage = (evt) => {
            var parts = evt.data.trim().split("\t");
            if (parts.length >= 2) {
                updateUI(parseFloat(parts[0]), parseFloat(parts[1]));
            }
        };
    } else { websocket.close(); }
}
document.getElementById("btn_connect").onclick = connectWS;
</script>
</body>
</html>
)=====";

```

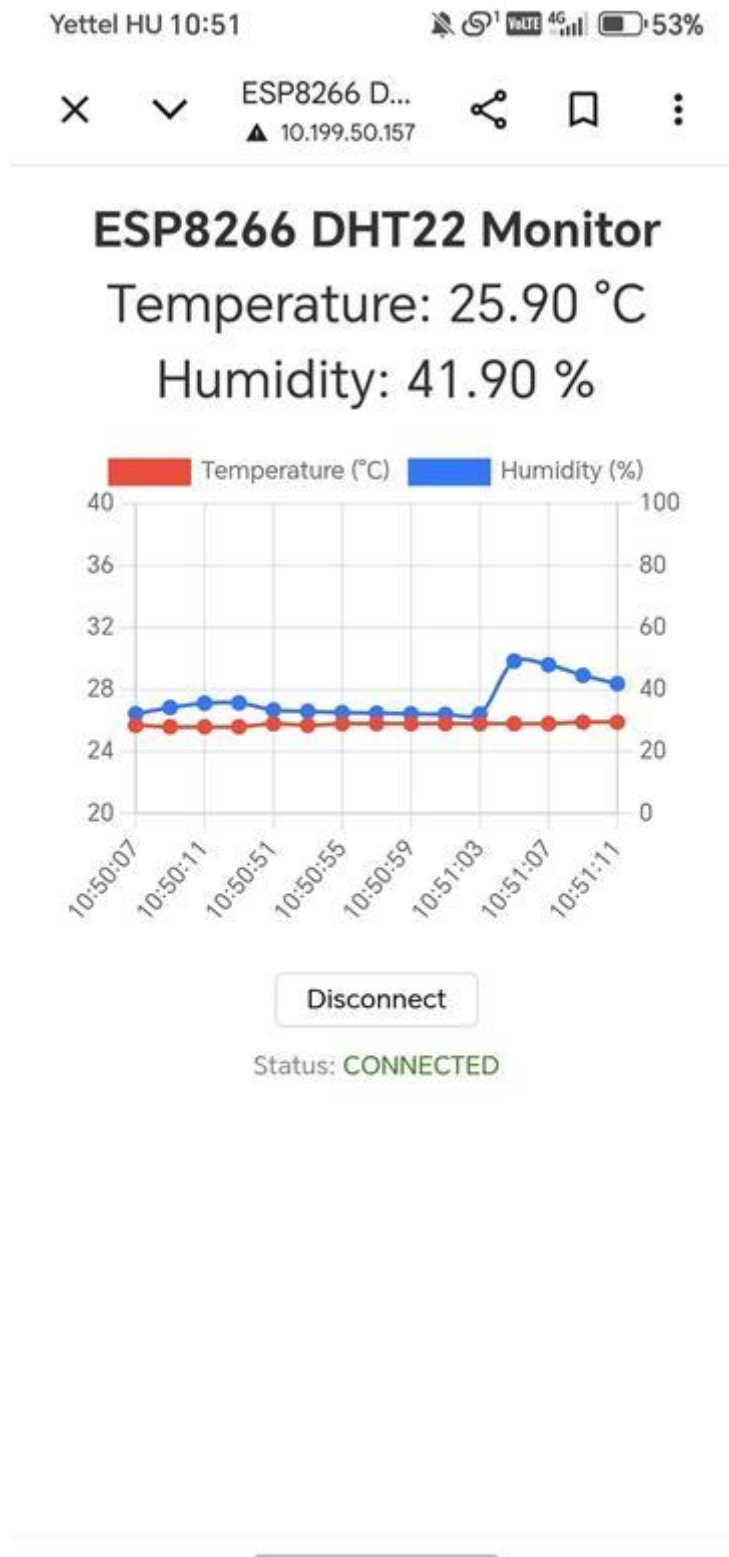
A projektről készült képek



(Az ESP8266 mikrovezérlő és a DHT22 szenzor összekötve)



(A program működésének ábrázolása grafikonon az Arduino IDE program Serial Plotter fülön, illetve írásos formában a Serial Monitoron)



(A program ábrázolása interneten keresztül telefonos eléréssel)

Fejlesztési lehetőségek

1. Adatok mentése (pl. felhő)
2. Több szenzor használata (pl. nyomás érzékelő)
3. Értesítés küldése a küszöbérték átlépésekor

Önreflexió

A projekt fejlesztése során nemcsak a WebSocket-kommunikációt és a valós idejű adatfeldolgozást tanultam meg hatékonyabban használni, hanem betekintést kaptam abba is, hogyan lehet hardveres méréseket modern webes megjelenítéssel kombinálni. Különösen hasznosnak tartom, hogy a rendszer rugalmasan bővíthető, így jó alapot nyújt további IoT-projektekhez és komplexebb adatvizualizációs feladatokhoz. A munka során fejlődött a problémamegoldó képességem, valamint a mikrokontroller programozási ismereteim is.